



UNIVERSITY OF DHAKA

**Conversational AI-Enhanced
Analysis of Retrieval Augmented
Generation for Honey-pot Data**

Submitted by

Exam Roll: 529

Registration No: II-130

Exam Roll: 520

Registration No: II-114

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS)



UNIVERSITY OF DHAKA

Conversational AI-Enhanced Analysis of Retrieval Augmented Generation for Honeypot Data

Submitted by

Exam Roll: 529

Registration No: H-130

Exam Roll: 520

Registration No: H-144

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS).

"Common sense is like deodorant. The people who need it most never use it."

Bill Murray

UNIVERSITY OF DHAKA

Abstract

This research presents an innovative framework that integrates honeypot-based cyber attack sensors with AI-powered Retrieval-Augmented Generation (RAG) technology and intelligent chatbot assistants to enhance cybersecurity threat analysis and automate incident response processes. The proposed system addresses the critical challenge of analyzing vast volumes of honeypot data manually, which is time-consuming and prone to human error. By leveraging Large Language Models (LLMs), natural language processing, and advanced visualization techniques, this research contributes to the development of a comprehensive cybersecurity platform that provides real-time threat detection, automated log analysis, and actionable intelligence through an interactive dashboard and chatbot interface. The system demonstrates significant improvements in threat detection accuracy, response time reduction, and overall security posture enhancement for organizations facing evolving cyber threats

Contents

Declaration Form	i
Abstract	iii
Acknowledgements	iv
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Introduction	1
1.1.1 The Growing Threat of Cyberattacks	1
1.1.2 Honeypots as Early Warning Systems	2
1.1.3 Classification of Honeypots	2
1.1.4 Cowrie Honeypot Log Data: Formats and Applications	3
1.1.5 MITRE ATT&CK TTP Mapping	5
1.2 Challenges in Honeypot Log Analysis	5
1.3 Leveraging Large-Language Models for Log Understanding	7
1.4 Problem Statement	8
1.5 Motivation	9
1.6 Research Questions	10
1.7 Contributions	11
1.8 Organization of the Project Report	13
2 Related Works	14
2.1 Honeypot Technology: Evolution and Current State	14
2.1.1 Traditional Honeypot Systems	14
2.2 AI-Enhanced Adaptive Honeypots	15
2.3 Large Language Models in Cybersecurity	16
2.3.1 LLM Applications and Capabilities	16
2.3.2 Limitations and Challenges	16
2.4 Retrieval-Augmented Generation (RAG) Systems	17
2.4.1 RAG in Cybersecurity Context	17
2.5 Conversational AI and Chatbot Systems	18
2.5.1 Cybersecurity-Focused Chatbots	18

2.6	MITRE ATTCK Framework Integration	18
2.6.1	Framework Benefits and Applications	18
2.7	Research Gaps and Limitations in Existing Works	19
2.7.1	Identified Gaps	19
2.8	Summary and Research Positioning	20
3	Proposed Methodologies	21
3.1	Introduction	21
3.2	System Model and Assumptions	21
3.2.1	Deployment Environment	21
3.2.2	Core Entities	22
3.2.3	Problem Formalization	22
3.3	Architecture Components	23
3.3.1	Data Collection in Ingestion Layer	24
3.3.1.1	Honeypot Deployment and Log Sources	24
3.3.1.2	HTTP Range and File-Tail Streaming	24
3.3.1.3	Parquet Archive Ingestion	25
3.3.2	State Persistence and Utility Modules	25
3.3.2.1	JSON Line Tracking and Byte-Offset Management	25
3.3.2.2	Date/Field Filters for Batch Loads	26
3.3.2.3	Geolocation Caching	26
3.3.3	Core Event Processing in Enrichment Layer	27
3.3.3.1	CowrieJSONProcessor: Session Reconstruction	27
3.3.3.2	Anomaly-Flagging Engine	28
3.3.3.3	MITRE ATT&CK TTP Mapping	28
3.3.4	Semantic Normalization Service	29
3.3.4.1	LLMCommandAnalyzer and Prompt Engineering	29
3.3.4.2	Batch Processing and Caching	29
3.3.4.3	Post-Processing with Bashlex	30
3.3.5	Vectorization Framework in Vectorization and Persistence Layer	30
3.3.5.1	Document Creation & Chunking	31
3.3.5.2	Embedding Model and Batching	31
3.3.5.3	FAISS IVF Index Training	31
3.3.5.4	Index Upgrade & Merge Utility	31
3.3.6	Metadata Storage & Analytics Engine	32
3.3.6.1	DuckDB Schema & Incremental Appends	32
3.3.6.2	SQL Explorer	33
3.3.7	Interfaces in Presentation Layer	34
3.3.7.1	Streamlit RAG Q&A	34
3.3.7.2	Geospatial Attack Map	35
3.3.8	Orchestration, Configuration & Deployment	35
3.3.8.1	Environment Variables & CLI Flags	36
3.3.8.2	Resource Scheduling	36
3.3.8.3	Monitoring & Resilience	36

3.4	System Integration and Dataflow	37
3.4.1	Discussion	38
3.5	Conclusion	38
4	Result Analysis	39
4.1	Improvement over Traditional Honeypot Log Analysis	39
4.2	Natural Language Querying with RAG Q&A	40
4.3	Structured Log Exploration Using SQL Table Analytics	42
4.4	Attack Map and Geospatial Visualization	45
4.5	Comparative Analysis	48
4.6	Summary of Results	51
4.7	Conclusion	51
5	Conclusions	53
5.1	Summary of Work	53
5.2	Limitations	55
5.3	Future Direction	57
	Bibliography	59

List of Figures

3.1	Architecture Diagram	23
4.1	Query to find last ip of attack	41
4.2	Query to find most used passwords	42
4.3	Query to find IP related to certain attack	43
4.4	SQL Analytic Table	44
4.5	Example Fields of SQL Analytics Table	45
4.6	Geospacial Visualization on Attack Map	46
4.7	Pie Chart of Attack Distribution	47
4.8	Detailed Attack Table	47
4.9	Raw Log	49
4.10	Chatbot Result	50

List of Tables

2.1	Advantages and Disadvantages of Traditional Honeypot Systems . .	15
2.2	Comparative Analysis - AI vs Traditional Honeypots	16
2.3	LLM Architecture Comparison for Cybersecurity	16
2.4	Comparative Analysis of Weakness	20
4.1	Comparative Analysis of Traditional vs. AI-Enhanced Honeypot Log Review	40

Chapter 1

Introduction

1.1 Introduction

1.1.1 The Growing Threat of Cyberattacks

The cybersecurity landscape has become increasingly perilous as adversaries deploy automated tools, exploit toolkits, and sophisticated malware to breach systems at unprecedented speed. According to IBM's 2023 Data Breach Report, the average time to identify a breach is 207 days, with an additional 70 days required for containment. Meanwhile, ransomware attacks now complete in under four days from initial compromise to full encryption, compared to two months pre-pandemic. The International Monetary Fund reports that global losses from cyberattacks have quadrupled since 2017, reaching an estimated USD 2.5 billion in 2024. These figures underscore the urgent need for faster, more effective detection and analysis methods to prevent attackers from achieving their objectives before conventional defenses can respond.

1.1.2 Honeypots as Early Warning Systems

Honeypots serve as deceptive traps that emulate real production assets, luring attackers into controlled environments where their actions can be observed and recorded without jeopardizing genuine infrastructure. By capturing detailed logs of each interaction—IP addresses, timestamps, authentication attempts, executed commands, and file transfers—honeypots provide invaluable intelligence on adversary tactics, techniques, and procedures (TTPs). This intelligence can inform threat-hunting efforts, guide security policy refinement, and accelerate incident response. However, the volume and complexity of honeypot logs demand automated analysis pipelines to extract actionable insights in near real time. [3]

1.1.3 Classification of Honeypots

Honeypots are generally categorized by the level of interaction they afford attackers:

1. Low-Interaction Honeypots

These systems provide minimal services or protocol emulation, limiting attacker access to simple, scripted responses.[8] While easy to deploy and requiring low maintenance, they capture only basic indicators of scanning or probing activities and yield low-volume data that often lack contextual depth.

2. Medium-Interaction Honeypots

Operating between low and high interaction, these honeypots emulate more complex service behaviors (e.g., partial shell access or vulnerable application endpoints).[1] They strike a balance between deployment effort and data richness but can be more readily identified by skilled adversaries once behavioral patterns are learned.

3. High-Interaction Honeypots

These honeypots offer attackers full access to a real operating system or application environment, typically via technologies such as ContainerSSH or virtual

machines.[9] High-interaction honeypots capture comprehensive TTPs—root commands executed, privilege escalations performed, lateral movement attempts, and exfiltration commands. The resulting data is exceptionally valuable for threat intelligence but demands advanced analysis tools to process at scale.

1.1.4 Cowrie Honeypot Log Data: Formats and Applications

Cowrie is a widely adopted, high-interaction SSH and Telnet honeypot that records every facet of an attacker's session in structured JSON format. Each log entry corresponds to a discrete event—connection establishment, authentication success or failure, command execution, file download, or session closure—with the following key fields[3]:

- **timestamp** (ISO 8601 string): The precise time of the event, enabling temporal correlation across sessions.
- **session** (string): A unique identifier for each SSH/Telnet session, used to reconstruct attacker workflows and calculate dwell times.
- **eventid** (string): The event type, such as `cowrie.session.connect`, `cowrie.login.success`, `cowrie.command.input`, or `cowrie.session.closed`.
- **src_ip**, **dst_ip**, **src_port**, **dst_port**: Network metadata capturing attacker origin and targeted port numbers.
- **username** and **password**: Credentials attempted by the adversary—critical for identifying brute-force and credential-spray campaigns.
- **input**: The exact string of each command the attacker issued, often including chained or obfuscated sequences.

- **outfile, url, sha256, size:** For `cowrie.session.file_download` events, these fields document files fetched by attackers, enabling analysis of dropper binaries or malware installers.
- **hassh, kexAlgs, encCS:** For key-exchange (`cowrie.client.kex`) events, Cowrie logs cryptographic parameters and host key hashes—data used to fingerprint malicious SSH clients and detect custom toolkits.

Because Cowrie logs every I/O exchange in JSON lines, they can be seamlessly streamed, filtered, and ingested by Python or Spark pipelines for near-real-time enrichment. In our implementation, we leverage these structured fields to:

1. **Reconstruct Threat Narratives:** Session IDs group related commands and downloads into coherent attacker stories, revealing multi-stage intrusion attempts.
2. **TTP Extraction:** Raw input strings are normalized via LLM prompting into concise, standardized descriptions of attacker techniques.
3. **Temporal Analysis:** Timestamps facilitate clustering of automated vs. manual sessions, as well as identification of high-velocity attack waves.
4. **Credential Analytics:** Username/password outcomes highlight weak-password targets and credential reuse across sessions.
5. **Malware Profiling:** Download metadata (URL, SHA-256) supports further sandbox analysis of fetched binaries.

By treating Cowrie's JSON-L format as a rich, schema-defined log source, our pipeline automates the transformation from raw telemetry to semantically enriched documents—laying the foundation for rapid retrieval, analytics, and threat hunting.

1.1.5 MITRE ATT&CK TTP Mapping

The MITRE ATT&CK® framework standardizes adversary behaviors into tactics (goals such as Reconnaissance or Exfiltration) and techniques (specific actions like Account Discovery or Ingress Tool Transfer).

Use Cases:

1. **Technique-Based Retrieval:** Query for all events associated with a given ATT&CK technique (e.g. “show every T1059 command”).
2. **Tactic Heatmaps:** Aggregate events by high-level tactic to reveal predominant attacker objectives across sessions.
3. **Threat Reporting:** Export technique frequencies to MITRE’s ATT&CK Navigator for visual threat modeling.

1.2 Challenges in Honeypot Log Analysis

Despite their efficacy in capturing attacker behavior, high-interaction honeypots generate volumes of complex, heterogeneous logs that pose significant analytic challenges:

1. Sheer Volume and Velocity

- A single high-interaction honeypot can record tens of thousands of events per day, spanning session lifecycle notifications, authentication attempts, command inputs, file downloads, and cryptographic key-exchange details.
- Traditional batch processing or manual reviews cannot keep pace with this continuous stream, leading to delayed or missed insights.

2. Heterogeneous Data Formats

- Cowrie logs mix structured JSON fields (timestamps, IP metadata, event IDs) with free-form command strings and binary protocol parameters.

- Parsers must handle diverse record types (`cowrie.session.connect`, `cowrie.command.input`, `cowrie.session.file_download`, etc.) with custom extraction logic, raising development and maintenance overhead.

3. Variable Command Syntax

- A single attacker objective (e.g., retrieving `/etc/passwd`) can be executed with myriad syntactic variants—environment-variable indirection, chained pipes, directory traversal, and encoding techniques.
- String-based searches or static rule-sets struggle to consolidate these variants, obscuring true technique prevalence.

4. Contextual Correlation Across Events

- Individual command entries lack standalone meaning; analysts must reconstruct session timelines, correlate login success/failure patterns, and aggregate download events to understand multi-stage intrusion workflows.
- Absent robust session mapping, critical context may be lost when events are viewed in isolation.

5. Semantic Ambiguity

- Commands that appear benign (e.g., `ls`, `cat`) may form part of reconnaissance or data-exfiltration when chained or combined with subsequent malicious steps.
- Automated anomaly detectors based solely on regex or statistical outliers either generate excessive false positives or overlook subtle malicious sequences.

6. Scalability of Endpoint Correlation

Analysts require rapid cross-correlation between honeypot-captured TTPs and live production logs to identify re-use of attacker commands on real servers—an effort demanding high-performance retrieval infrastructures.

Together, these challenges underscore the necessity for an automated, semantically-aware pipeline—combining session reconstruction, anomaly flagging, LLM-driven normalization, and vectorized retrieval—to transform raw honeypot telemetry into real-time, actionable threat intelligence.

1.3 Leveraging Large-Language Models for Log Understanding

Large-Language Models (LLMs) such as GPT-4, LLaMA-3, and other transformer-based architectures have revolutionized natural-language processing by demonstrating near-human performance on tasks like summarization, intent extraction, and context-aware generation.[8] Their capabilities address several honeypot log analysis challenges:

1. **Semantic Normalization of Commands** LLMs can parse obfuscated or chained bash commands and produce concise, plain-English descriptions of attacker intent (e.g., “This command enumerates local user accounts”)—abstracting away variable syntax and reducing semantic variability.
2. **Contextual Reasoning Across Events** Through chain-of-thought prompting, LLMs can reason step-by-step about session timelines—linking login events, file downloads, and subsequent privilege escalations into coherent threat narratives.
3. **Reducing False Positives in Anomaly Detection** By analyzing normalized descriptions rather than raw strings, LLMs help distinguish innocuous system maintenance commands from genuine malicious actions, lowering reliance on brittle regex rules.
4. **Grounded Summarization via Retrieval-Augmented Generation (RAG)** Combining LLMs with vector-based retrieval (e.g., FAISS) allows system queries to be “grounded” in actual log content by first retrieving the top-N semantic

matches. These are then used to generate answers or summarize by passing them as context to the LLM to mitigate hallucinations and ensure factual accuracy.[1]

5. **Scalability and Automation** Batch LLM analyses of large sets of commands enable automated enrichment at scale, while caching mechanisms reduce API costs. Incremental processing pipelines can integrate LLM-driven normalization without human intervention, enabling near-real-time understanding of emerging attacker techniques.

By embedding LLM-driven semantic analysis into our honeypot pipeline, we transform raw logs into structured, normalized TTPs—facilitating efficient vector retrieval, precise threat hunting, and on-demand natural-language querying of adversary behaviors.

1.4 Problem Statement

Contemporary security operations face a critical gap between the richness of high-interaction honeypot telemetry and the timeliness of actionable threat intelligence. Specifically:

- **Data Overload:** Honeypots like Cowrie generate tens of thousands of JSON-encoded events per day, encompassing session metadata, authentication attempts, free-form command inputs, file-download logs, and cryptographic exchanges.
- **Heterogeneous Formats:** The mixture of structured fields (timestamps, IP addresses, event IDs) and unstructured command strings demands custom parsers and disparate analysis tools, leading to brittle, costly pipelines that are difficult to maintain.
- **Semantic Variability:** Attackers employ obfuscation—aliasing, environment-variable indirection, chained commands, and encoded payloads—to mask their techniques. Simple regex or n-gram based anomaly detection either

produces excessive false positives or fails to unify these variants under common TTPs.

- **Slow Feedback Loop:** Manual or batch-oriented analyses introduce delays measured in days or weeks, by which time adversaries may have completed ransomware campaigns or established footholds in production systems.

These shortcomings culminate in a reactive posture, where defenders struggle to interpret vast logs quickly enough to thwart ongoing attacks or to trace attacker movements in real time.

1.5 Motivation

To address this urgent need for faster, semantic understanding of honeypot data, we propose an automated, end-to-end pipeline that seamlessly bridges raw telemetry and real-time human-readable insights. Our motivation is driven by:

(a) Accelerated Detection and Response

Converting raw logs into normalized TTP descriptions via LLMs and retrieving relevant events in under two seconds empowers security teams to investigate and contain threats within hours rather than days.

(b) Semantic Cohesion

By abstracting diverse command syntaxes into unified, plain-English descriptions, defenders can quickly identify recurring attacker behaviors regardless of obfuscation techniques.

(c) Scalable Intelligence

Embedding normalized TTPs into a vector search index (IVF-FAISS) enables efficient similarity queries over millions of events, supporting both ad hoc investigations and continuous monitoring.

(d) **Standards Alignment**

Mapping events to the MITRE ATT&CK framework ensures that the extracted intelligence aligns with industry-recognized TTP taxonomies, facilitating threat sharing and standardized reporting.

(e) **Operational Practicality**

Leveraging open-source components (Cowrie, Python, LangChain, FAISS, DuckDB) and public LLM APIs (e.g., GPT-4 turbo) delivers a reproducible, cost-effective solution that can be adopted by organizations of varying size and maturity.

By unifying session reconstruction, anomaly flagging, LLM-driven normalization, semantic embedding, and retrieval-augmented interfaces, our pipeline transforms static honeypot telemetry into a dynamic, actionable asset for proactive cyber defense.

1.6 Research Questions

Building on the problem statement and motivations, this work investigates the following key research questions:

RQ1: How can semantic normalization of honeypot command logs via Large-Language Models improve the consistency and interpretability of attacker TTP descriptions?

Subquestion: What prompt engineering strategies yield the most reliable, concise command summaries across diverse obfuscated syntax?

RQ2: To what extent does embedding LLM-normalized TTP descriptions in a FAISS IVF index enhance similarity retrieval performance compared to raw command embeddings?

Subquestion: How do retrieval precision and recall vary with and without LLM-driven normalization?

RQ3: Can real-time RAG (Retrieval-Augmented Generation) queries over honeypot telemetry meet sub-2-second latency requirements without sacrificing factual accuracy?

Subquestion: What caching strategies (exact and semantic) best reduce API calls and response times?

RQ4: How effectively do regex-based anomaly flags and MITRE ATT&CK mappings facilitate high-level filtering and tactical aggregation of attacker events?

Subquestion: Which subset of anomaly patterns correlates most strongly with LLM-identified malicious intents?

RQ5: What deployment and scalability considerations arise when integrating incremental streaming ingestion, batch vector updates, and index retraining in a production environment?

Subquestion: How do GPU scheduling, DuckDB metadata storage, and FAISS index parameters (e.g., nlist) impact throughput and resource utilization?

Together, these research questions drive the design, implementation, and evaluation of an LLM-augmented honeypot log analysis pipeline that transforms vast raw telemetry into real-time, actionable threat intelligence.

1.7 Contributions

Novelty of the Current Research

While many log-analysis frameworks rely on batch processing and isolated tools, our work presents a fully integrated pipeline that transforms high-interaction honeypot telemetry into live, semantically enriched knowledge—enabling real-time threat exploration without manual intervention. The key contributions of this research are:

- Incremental real-time log streaming and state management: We track byte offsets and line hashes to ingest new Cowrie JSON logs continuously

from HTTP(S) streams or local files—even across log rotations—without reprocessing old data. This streaming mechanism underpins automated, production-grade enrichment and vector updates, ensuring our system always operates on the latest attacker activity.

- Unified session-aware enrichment engine: A single, optimized parser reconstructs each attacker session end-to-end, propagating the most recent successful credentials to every event record. Every log entry is enriched with IP geolocation, regex-based anomaly flags, and MITRE ATT&CK technique mappings within one coherent module, eliminating the complexity of multi-stage tool chains.
- Large-scale batch and parallel vectorization: Our batch tool loads Parquet logs in configurable row-chunks, applies filters by date, session, or event type, and processes millions of events in parallel threads. Chunk size, embedding batch size, and FAISS clustering parameters are all user-tunable at runtime, enabling on-the-fly scaling to match data volumes without any code changes.
- FAISS IVF upgrade & merge utility: We provide a command-line script that merges or retrains IVF-Flat indexes across multiple vectorstore shards. By re-embedding all document texts in bulk and producing a unified, LangChain-compatible FAISS vectorstore, this utility solves the practical challenge of scaling vector indexes as new logs accumulate over extended deployments.
- Open-source, fully configurable pipeline: Every component—log processor, real-time streaming service, batch vectorizer, index merger, and web UI—is implemented in Python, accepts environment variables or CLI arguments for all parameters, and persists its state to JSON or DuckDB. This end-to-end reproducible workflow can be deployed on commodity Linux servers or cloud VMs, making advanced honeypot log analysis accessible without proprietary platforms.

Together, these contributions deliver a novel, semantically indexed honeypot analysis platform that empowers security teams to query, visualize, and explore attacker behaviors with unprecedented speed and depth.

1.8 Organization of the Project Report

This report is organized as follows:

- **Chapter 2:** Discusses related work on honeypot analytics, conversational AI, and vector search technologies.
- **Chapter 3:** Presents the proposed system design, methodologies, and algorithms.
- **Chapter 4:** Details system implementation including data pipeline, conversational interface, and visualization components.
- **Chapter 5:** Evaluates system with experiments on performance, accuracy, and usability.
- **Chapter 6:** Discusses the insights gained, limitations, and outlines future research directions.
- **Chapter 7:** Concludes with an overall summary of contributions and final remarks.

Chapter 2

Related Works

This chapter presents a comprehensive review of existing literature and recent developments in the field of cybersecurity, focusing on honeypot technology, Large Language Models (LLMs) in cybersecurity applications, Retrieval-Augmented Generation (RAG) systems, and conversational AI interfaces for threat intelligence. The review examines related concepts, existing design approaches, and experimental observations that form the foundation of this research while highlighting comparative discussions on the pros and cons of existing works.

2.1 Honeypot Technology: Evolution and Current State

2.1.1 Traditional Honeypot Systems

Honeypots have evolved significantly since their introduction in the early 2000s, serving as decoy systems designed to attract and analyze cyber attackers. According to the comprehensive review by Moric et al. (2025)[6], honeypots are classified based on their interaction levels: low-interaction honeypots that simulate basic services, medium-interaction honeypots providing

controlled environments, and high-interaction honeypots offering complete operational systems. Recent systematic analysis by Kumar et al. (2025)[4] categorizes honeypots into production honeypots for real-time threat detection and research honeypots for comprehensive threat intelligence gathering. The fundamental architecture involves decoy systems, captor functions for monitoring and profiling, and response mechanisms for threat containment

Pros	Cons
Low resource consumption	Static nature
Minimal false positives	Limited scope
Early warning capabilities	Resource-intensive
Effective deception and attack deflection	Risk of compromise

TABLE 2.1: Advantages and Disadvantages of Traditional Honeypot Systems

2.2 AI-Enhanced Adaptive Honeypots

The integration of artificial intelligence has revolutionized honeypot technology. Kareem et al. (2024)[1] demonstrated that AI-driven adaptive honeypots can increase attacker interaction time by 40% (from 32 to 45 minutes average) while collecting 50% more actionable data compared to static systems. These systems achieve 90% detection rates versus 65% for traditional honeypots, though at a 30% higher resource cost. Recent work by Wang et al. (2025)[12] introduced HoneyGPT, which breaks the traditional trilemma between deception capability, interaction depth, and system flexibility through LLM-powered prompt engineering and Chain-of-Thought (CoT) strategies. Their system outperformed traditional Cowrie honeypots in deception effectiveness while maintaining long-term dialogue consistency.

Aspect	Traditional Honeypots	AI-Enhanced Honeypots
Detection Rate	65-75%	85-95%
Attacker Engagement	Static, limited	Dynamic, adaptive
Resource Usage	Low	Moderate to High
False Positive Rate	Moderate	Low
Setup Complexity	Medium	High
Maintenance	Manual	Semi-automated

TABLE 2.2: Comparative Analysis - AI vs Traditional Honeypots

2.3 Large Language Models in Cybersecurity

2.3.1 LLM Applications and Capabilities

The systematic literature review by Otal et al. (2024)[7] identifies over 300 works spanning 25+ LLMs across 10+ cybersecurity scenarios. LLMs demonstrate significant potential in threat intelligence processing, vulnerability detection, and automated security analysis. Arikkat et al. (2024)[2] developed IntellBot, demonstrating that LLM-based chatbots using RAG achieve high accuracy scores (BERT: 0.8, cosine similarity: 0.81) for cybersecurity knowledge delivery. Their system successfully integrates diverse data sources including 2,447 PDFs, 7,989 JSON files from VirusTotal, 197,256 NVD vulnerabilities, and 21,825 blog posts.

Model	Parameters	Context Window	Cybersecurity Performance	Cost
GPT-4	175B	8k-128k	High	Commercial
LLaMA-3	8B-70B	8k	High	Open Source
Claude-3	60B-200B	200k	Very High	Commercial
Mistral	3B-22B	8k	Good	Open Source

TABLE 2.3: LLM Architecture Comparison for Cybersecurity

2.3.2 Limitations and Challenges

Despite promising results, LLMs face significant challenges in cybersecurity applications:

- (a) **Hallucination Issues:** LLMs may generate false or outdated security information

- (b) **Knowledge Cutoff Limitations:** Training data becomes outdated, missing recent threats
- (c) **Domain Specificity:** General LLMs lack specialized cybersecurity knowledge
- (d) **Contextual Understanding:** Limited ability to maintain context in complex security scenarios

2.4 Retrieval-Augmented Generation (RAG) Systems

2.4.1 RAG in Cybersecurity Context

RAG technology addresses LLM limitations by combining retrieval mechanisms with generative capabilities. The approach involves two main components: a retriever that searches external knowledge bases and a generator that produces contextually appropriate responses. Recent research by Lanka et al. (2024)[5] successfully deployed RAG-based systems for real-time threat analysis, achieving 75% cosine similarity thresholds for semantic matching between attacker commands and threat intelligence. Their system processes high-interaction honeypot logs in AWS/GCP environments with minimal latency. Singh and Agrawal (2024)[11] demonstrated that RAG enables scalable, proactive threat detection through improved identification of phishing and emerging malware using unstructured data sources. However, their work lacks live system implementation and real-time capabilities.

RAG Implementation Challenges:

Retrieval accuracy depends on knowledge base quality and completeness

- Context window limitations affect long-form threat analysis
- Real-time performance constraints in high-volume environments
- Integration complexity with existing security infrastructure

- (b) **Knowledge Cutoff Limitations:** Training data becomes outdated, missing recent threats
- (c) **Domain Specificity:** General LLMs lack specialized cybersecurity knowledge
- (d) **Contextual Understanding:** Limited ability to maintain context in complex security scenarios

2.4 Retrieval-Augmented Generation (RAG) Systems

2.4.1 RAG in Cybersecurity Context

RAG technology addresses LLM limitations by combining retrieval mechanisms with generative capabilities. The approach involves two main components: a retriever that searches external knowledge bases and a generator that produces contextually appropriate responses. Recent research by Lanka et al. (2024)[5] successfully deployed RAG-based systems for real-time threat analysis, achieving 75% cosine similarity thresholds for semantic matching between attacker commands and threat intelligence. Their system processes high-interaction honeypot logs in AWS/GCP environments with minimal latency. Singh and Agrawal (2024)[11] demonstrated that RAG enables scalable, proactive threat detection through improved identification of phishing and emerging malware using unstructured data sources. However, their work lacks live system implementation and real-time capabilities.

RAG Implementation Challenges:

Retrieval accuracy depends on knowledge base quality and completeness

- Context window limitations affect long-form threat analysis
- Real-time performance constraints in high-volume environments
- Integration complexity with existing security infrastructure

2.5 Conversational AI and Chatbot Systems

2.5.1 Cybersecurity-Focused Chatbots

The development of specialized cybersecurity chatbots represents a growing trend. Simoni et al. (2024)[10] developed MoRSE, a dual RAG architecture chatbot that outperformed GPT-4 by 10-50% in correctness across cybersecurity questions, achieving 84% accuracy on CVE detection versus 34% by GPT-4.

The NIST National Cybersecurity Center of Excellence (NCCoE) recently developed an internal RAG-based LLM chatbot for cybersecurity guidelines discovery and summarization, demonstrating institutional adoption of these technologies.

Key Features of Modern Cybersecurity Chatbots:

- Multi-source knowledge integration (CVE, MITRE ATT&CK, threat feeds)
- Real-time threat intelligence updates
- Context-aware response generation
- Integration with existing security tools (SIEM, SOC platforms)

2.6 MITRE ATT&CK Framework Integration

2.6.1 Framework Benefits and Applications

The MITRE ATT&CK framework provides structured adversary behavior knowledge that enhances cybersecurity systems. Research shows that RAG systems significantly improve TTP interpretation by providing relevant context from the framework.

Key Benefits:

Standardized threat taxonomy for consistent analysis

- Real-world attack pattern documentation
- Enhanced threat hunting capabilities
- Improved security control mapping

Implementation Challenges: Framework complexity requires specialized knowledge

- Continuous updates needed to maintain relevance
- Resource-intensive comprehensive implementation
- Integration difficulties with legacy systems

2.7 Research Gaps and Limitations in Existing Works

2.7.1 Identified Gaps

Integrated System Architecture: Most research focuses on individual components rather than comprehensive integrated systems combining honeypots, LLMs, and RAG.

- **Real-time Performance:** Limited evaluation of system performance under high-volume, real-time conditions.
- **Scalability Assessment:** Insufficient analysis of scalability for enterprise deployment.
- **User Experience:** Minimal focus on user interface design and analyst workflow integration.
- **Cost-Benefit Analysis:** Limited economic evaluation of implementation costs versus security benefits.

Research Area	Weaknesses	Gap Addressed by This Work
Traditional Honeypots	Static, easily detected	Dynamic AI-powered adaptation
AI Honeypots	High complexity, limited integration	Streamlined integration pipeline
LLM Cybersecurity	Hallucination, outdated knowledge	RAG-enhanced accuracy
RAG Systems	Implementation complexity	User-friendly interface design
Cybersecurity Chatbots	Limited domain knowledge	Honeypot-specific intelligence

TABLE 2.4: Comparative Analysis of Weakness

2.8 Summary and Research Positioning

This literature review reveals a clear evolution from static honeypot systems toward intelligent, adaptive security solutions. While significant progress has been made in individual technologies—AI-enhanced honeypots, LLM-based cybersecurity applications, and RAG systems—there remains a critical gap in comprehensive integration of these technologies into unified, user-friendly platforms.

The proposed research addresses these limitations by developing an integrated system that combines:

- Real-time honeypot data processing with AI-powered analysis
- RAG-enhanced LLM capabilities for accurate threat intelligence
- Intuitive conversational interfaces for security analysts
- Comprehensive dashboard visualization for strategic oversight

This work positions itself at the intersection of these evolving technologies, contributing to the advancement of proactive cybersecurity defense mechanisms while addressing the practical needs of security operations centers and threat intelligence teams.

Chapter 3

Proposed Methodologies

3.1 Introduction

This chapter provides a detailed exposition of the proposed system's methodology. Each architectural component is described—from the basic building blocks (data ingestion, parsing, enrichment, vector storage, and analytics) to the integration into a unified threat intelligence and question-answering platform. The logical flow allows both technical clarity and traceability of design.

3.2 System Model and Assumptions

The proposed pipeline ingests attacker interactions captured by high-interaction honeypots and transforms them into enriched, indexable vectors and metadata records for real-time queries, analytics, and visualization.

3.2.1 Deployment Environment

- Honeypot Sensor
 - Ubuntu 22.04 VM running Cowrie, exposed to the Internet via a public IP address for adversary attraction.
 - Outputs JSON-encoded logs to a local directory or HTTP endpoint.

- **Processing Host**
 - Windows 10 Professional Desktop
 - 16 GB RAM, AMD Ryzen 5 5600X (6 cores), AMD Radeon RX 6600 GPU

3.2.2 Core Entities

- **Honeypot Sensors:** Data sources generating raw Cowrie JSON events.
- **Ingestion Layer:** Continuously reads new log events via HTTP Range requests or file tailing.
- **Enrichment Layer:** Parses events, reconstructs sessions, and augments records with geolocation, anomaly flags, and MITRE ATT&CK TTP mappings.
- **Vectorization & Persistence Layer:** Converts enriched events into fixed-dimensional vectors and stores both vectors and metadata.
- **Presentation Layer:** Supports Retrieval-Augmented Q&A, SQL-based analytics, and geospatial visualizations.

3.2.3 Problem Formalization

Let $E = \{e_i\}$ be the time-ordered sequence of raw honeypot events. For each event e_i , the pipeline computes:

- A vector embedding $v_i \in R^d$
- The metadata record m_i containing fields such as timestamp, session ID, anomaly flag, MITRE TTPs, and geolocation

We require a system supporting:

- (a) Ingest: $\text{ingest}(e_i) \rightarrow (v_i, m_i)$ with $O(1)$ incremental updates as new events arrive,

- (b) Query: $\text{query}(q) \rightarrow \{m_j\}$ by nearest-neighbor search in d -dimensional embedding space,
- (c) SQL Analytics: ad-hoc queries over $\{m_i\}$ to compute aggregations, filters, and counts.
- (d) Geospatial plotting: Visualize $(\text{lat}, \text{lon}) \subseteq m_i$ on interactive map

3.3 Architecture Components

The architectural diagram of our project is shown below in Figure 3.1. The

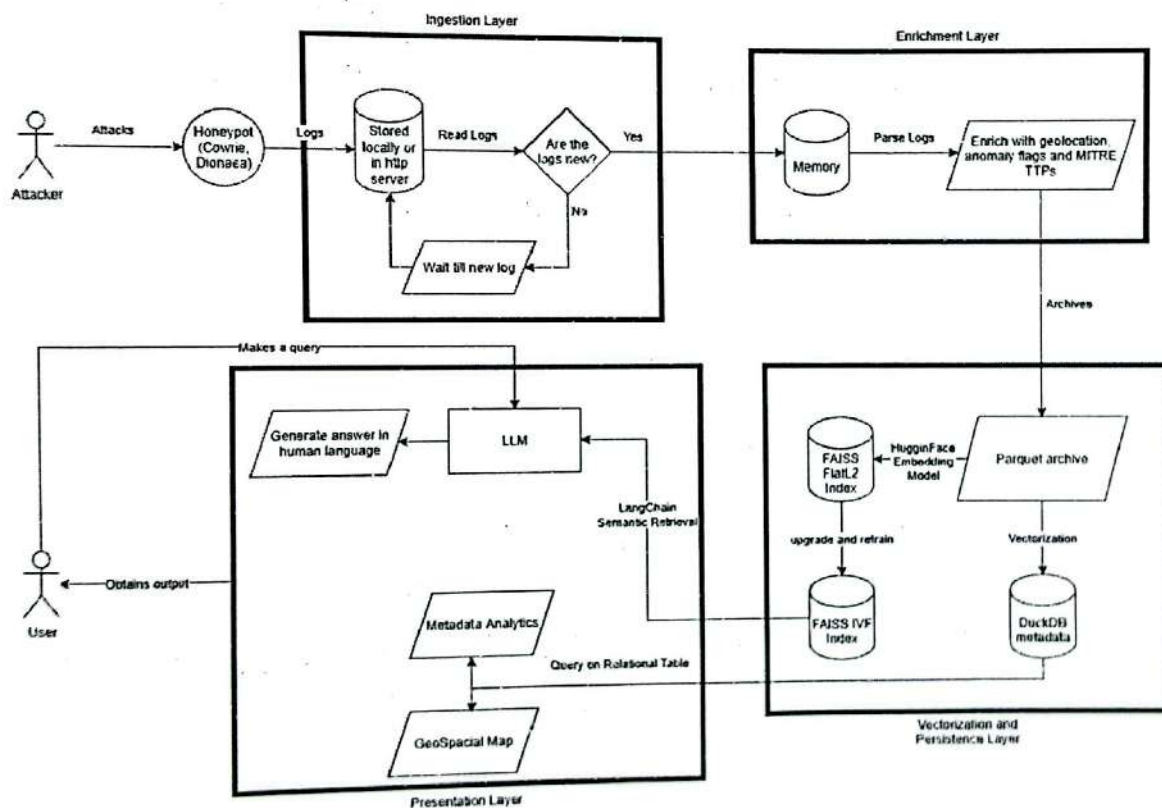


FIGURE 3.1: Architecture Diagram

complete architecture is described in a bottom-up approach.

3.3.1 Data Collection in Ingestion Layer

Reliable data collection is the foundation of our pipeline, ensuring that every attacker interaction captured by the honeypot is ingested without loss or duplication. This section describes our honeypot deployment, the mechanisms for streaming live JSON logs, and the batch ingestion of archived Parquet files.

3.3.1.1 Honeypot Deployment and Log Sources

We deploy high-interaction honeypots using Cowrie on a Ubuntu 22.04 virtual machine provisioned in a public-facing network segment. Each honeypot listens on SSH and Telnet ports, emulating real servers to attract adversaries. Cowrie is configured to record all session events—connection attempts, authentication successes or failures, command inputs, file downloads, and session closures—as line-delimited JSON files. Log files are written locally under `/var/log/cowrie/` and optionally exposed over HTTP by a lightweight web server for centralized collection.

3.3.1.2 HTTP Range and File-Tail Streaming

To ingest live log events with minimal latency, our ingestion service supports two streaming modes:

- **HTTP Range Requests:** When logs are exposed via HTTP, we issue periodic Range requests that specify a byte offset (`Range: bytes=<last_byte>{}`) to fetch only new data appended since the last poll.
- **Local File Tailing:** When logs reside on the same host or a shared filesystem, we tail the JSON file using Python's file seek API, tracking the last read offset.

In both modes, we maintain a persistent state file (`pipeline_state.json`) recording the most recent byte offset and line hashes. On each iteration, new lines are read, parsed into JSON objects, and passed to the enrichment layer.

If log truncation or rotation is detected (via a mismatched hash or file size reduction), we reset the offset to zero and resume ingestion without losing events.

3.3.1.3 Parquet Archive Ingestion

Historical log archives are stored as Parquet files partitioned by date under `processed_data/`. To backfill or reprocess data, our batch ingestion tool scans this directory for new Parquet files, ordering them by modification timestamp. Each file is read in configurable row-batches (default 10,000 rows) using PyArrow and Pandas. CLI flags allow filters by:

- Date or date range (e.g., `--start-date 2025-08-01 --end-date 2025-08-07`)
- Session ID list (`--session-ids ses123, ses456`)
- Event type (`--event-type cowrie.command.input`)

Filtered rows are converted to enriched event dictionaries and fed into downstream normalization and vectorization stages. This approach enables efficient backfills, re-indexing, and historical analyses without reprocessing the entire archive in a single run.

3.3.2 State Persistence and Utility Modules

Efficient and reliable processing of honeypot logs requires auxiliary modules that track ingestion progress, filter data for targeted reprocessing, and cache external enrichments. This section describes the key utilities that underpin the pipeline's resilience and performance.

3.3.2.1 JSON Line Tracking and Byte-Offset Management

To prevent reprocessing of already ingested events—especially across log rotations or pipeline restarts—the ingestion service maintains a persistent state file containing:

- The last byte offset read from each log source (HTTP endpoint or local file path)
- A rolling hash of the most recently processed line to detect file truncation or rotation

On startup, the service loads these values and seeks to the stored offset when reading. After processing each batch of lines, it atomically updates the state file with the new offset and hash. If the file size shrinks or the new hash does not match expectations, the offset is reset to zero, ensuring no events are silently skipped.

3.3.2.2 Date/Field Filters for Batch Loads

Batch ingestion of historical Parquet archives can involve large volumes of data. To support efficient backfills and selective reprocessing, the batch loader accepts command-line filters that apply at the Pandas DataFrame level:

- `--start-date` and `--end-date` (ISO 8601) restrict rows by timestamp
- `--session-ids` specifies a comma-separated list of session identifiers to include
- `--event-types` filters by Cowrie event IDs (e.g., `cowrie.command.input`, `cowrie.session.closed`)

These filters are evaluated before any downstream enrichment or normalization, minimizing memory use and speeding up targeted analyses.

3.3.2.3 Geolocation Caching

IP geolocation enrichments rely on third-party APIs that are subject to rate limits and latency. We implement an in-memory LRU cache keyed by source IP address:

- On first lookup, the IPInfo API is queried for country, region, city, ASN, and latitude/longitude

- The result is cached for a configurable TTL (default 24 hours)
- Subsequent events from the same IP retrieve data from cache, eliminating redundant API calls

This caching layer both accelerates enrichment and controls external API usage, ensuring consistent performance even under high ingestion rates.

3.3.3 Core Event Processing in Enrichment Layer

At the heart of our pipeline lies the `CowrieJSONProcessor`, which transforms raw JSON events into semantically enriched records ready for normalization and embedding. This section details session reconstruction, anomaly flagging, and MITRE ATT&CK mapping.

3.3.3.1 CowrieJSONProcessor: Session Reconstruction

The `CowrieJSONProcessor` class orchestrates parsing and stateful session aggregation:

- **Event Parsing:** Converts each JSON line into a Python dictionary, extracting core fields (`timestamp`, `session`, `eventid`, `input`, `username`, `password`, etc.).
- **Session State Tracking:** Maintains an in-memory map of active sessions keyed by session ID. For each session, it records:
 - Connection start and end timestamps to compute dwell time
 - Latest successful credentials to annotate subsequent events
 - Ordered list of commands and downloads for narrative reconstruction
- **Output Emission:** Upon receiving a `cowrie.session.closed` event, the processor flushes the complete session record—comprising all enriched events—to downstream queues for normalization and indexing.

3.3.3.2 Anomaly-Flagging Engine

To surface overtly suspicious behaviors, we apply over 120 regular expressions against each `cowrie.command.input` string. Patterns are categorized into:

- Destructive Operations (e.g., `rm -rf`, `chmod 777`)
- Reverse-Shell Primitives (e.g., `bash -i`, `nc -e`, `python -c 'import socket'`)
- Payload Decoding (e.g., `base64 -d`, `xxd -r`)
- Reconnaissance Commands (e.g., `whoami`, `netstat`, `ifconfig`)
- Privilege Escalation Attempts (e.g., `sudo su`, `chmod +s`)

If any pattern matches, the event's `anomaly_flag` is set to `True`; otherwise, it remains `False`. This boolean field enables quick filtering of high-risk commands and statistical anomaly trend analysis.

3.3.3.3 MITRE ATT&CK TTP Mapping

To align events with industry-standard threat taxonomies, we maintain a mapping of MITRE ATT&CK technique IDs to representative command patterns. During enrichment:

- The command string is lowercased and tokenized
- For each technique in our dictionary (e.g., T1087: account discovery—`whoami`, `id`; T1059: command scripting—`bash`, `perl`, `powershell`), we test for substring presence
- All matching technique IDs are aggregated into a JSON array under the `mitre_ttp` field

This structured mapping supports tactic-level aggregation (e.g., Reconnaissance vs. Defense Evasion), technique-based retrieval, and export to visualization tools such as ATT&CK Navigator.

3.3.4 Semantic Normalization Service

The Semantic Normalization Service translates raw, heterogeneous bash commands into concise, human-readable descriptions of attacker intent. It leverages LLMs for deep language understanding and post-processes outputs for consistency.

3.3.4.1 LLMCommandAnalyzer and Prompt Engineering

The LLMCommandAnalyzer class encapsulates interactions with chat-based LLMs (e.g., GPT-4-turbo). Key design elements include:

- **Role Definition:** In prompts, the model assumes the persona of a security analyst or Linux shell expert to frame its explanations.
- **Chain-of-Thought Guidance:** Sequential prompt templates encourage the model to first parse command structure, then identify functionality, and finally summarize intent.
- **Teaching Examples:** Each prompt includes two illustrative examples showing transformation of obfuscated commands into plain-English descriptions and corresponding MITRE technique IDs.
- **Output Schema Enforcement:** The final prompt instructs the model to respond in JSON with fields `intent_description`, `technique_id`, and `confidence_score`, facilitating downstream parsing.

3.3.4.2 Batch Processing and Caching

To optimize throughput and cost:

- **Batch Submission:** Commands are grouped (up to 50 per batch) into a single API call, reducing per-call latency and amortizing fixed token overhead.
- **Local Cache:** A TTL-based in-memory dictionary stores recent command inputs and their normalized responses. Before issuing an API call,

the analyzer checks the cache; cache hits return instantly, avoiding external calls and preserving API quota.

- **Error Retries:** Transient API failures trigger up to three exponential-backoff retries before marking the command as “unprocessed” for manual review.

3.3.4.3 Post-Processing with Bashlex

Raw LLM outputs may include specific filenames, URLs, or variable placeholders. We apply a Bashlex-based sanitizer to:

- **Strip Literal Arguments:** Remove file paths, IP addresses, and shell variables, replacing them with generic placeholders (e.g., <FILE>, <IP>) to focus descriptions on behavior rather than specifics.
- **Normalize Verbs:** Convert synonymous actions (e.g., “download” vs. “fetch”) to a standard lexicon used across all descriptions.
- **Ensure Consistent Tense:** Transform descriptions into present-tense, active-voice statements (e.g., “Enumerates user accounts” rather than “Enumerated users”).

This post-processing yields uniform, concise TTP descriptions optimized for embedding and retrieval.

3.3.5 Vectorization Framework in Vectorization and Persistence Layer

The vectorization framework’s function is to transform normalized TTP descriptions and session metadata into fixed-dimensional vectors and organize them into an efficient index for fast semantic similarity search.

3.3.5.1 Document Creation & Chunking

Each enriched event is encapsulated as a LangChain Document with its normalized TTP description in `page_content` and associated metadata (timestamp, session ID, anomaly flag, MITRE TTPs). To accommodate long descriptions and multi-command sessions, we apply a recursive chunking strategy using `RecursiveCharacterTextSplitter` with a 1,000-character chunk size and 50-character overlap. This ensures semantic coherence across chunks while respecting embedding model context limits.

3.3.5.2 Embedding Model and Batching

Vector representations are generated via the `sentence-transformers/all-MiniLM-L6-v2` model, producing 384-dimensional embeddings. Embedding is performed in GPU-accelerated batches of up to 512 chunks to maximize throughput. Batch size and GPU device selection are configurable via CLI flags, enabling adaptation to hardware constraints and desired performance.

3.3.5.3 FAISS IVF Index Training

To enable approximate nearest-neighbor retrieval over large vector collections, we build a FAISS `IndexIVFFlat` index with `nlist=256` clusters. The index is trained on a 10% sample of the embedding corpus to learn cluster centroids before adding the full set of embeddings. This quantized index structure provides sub-linear search times while maintaining high recall.

3.3.5.4 Index Upgrade & Merge Utility

As new data arrives and multiple index shards accumulate, the `faiss-ivf_upgrade.py` CLI tool supports ongoing index maintenance by:

- Loading existing LangChain FAISS vectorstores from disk
- Re-embedding document texts with the current embedding model to ensure consistency

- Retraining or merging individual FAISS `IndexIVFFlat` shards into a unified index
- Exporting the consolidated vectorstore for use by downstream semantic retrieval chains

This utility preserves index performance and coherence over long-term deployments.

3.3.6 Metadata Storage & Analytics Engine

The Metadata Storage & Analytics Engine persists enriched chunk metadata in a relational format and provides ad-hoc query capabilities for security analysts.

3.3.6.1 DuckDB Schema & Incremental Appends

We use DuckDB as an embedded analytical database to store metadata for each vector chunk. The primary table, `vector_metadata`, is defined with the following schema:

- `chunk_id` UUID PRIMARY KEY
- `timestamp` TIMESTAMP
- `session_id` VARCHAR
- `anomaly_flag` BOOLEAN
- `mitre_ttps` JSON
- `country` VARCHAR
- `region` VARCHAR
- `city` VARCHAR
- `asn` VARCHAR
- `latitude` DOUBLE
- `longitude` DOUBLE

- `intent_description` TEXT
- `technique_id` VARCHAR

After each embedding batch, a bulk insert operation appends new rows to `vector_metadata` using DuckDB's `INSERT INTO ... VALUES` with prepared statements. We enforce schema consistency via `PRAGMA table_info('vector_metadata')` on startup, automatically adding missing columns when the pipeline evolves. This incremental append strategy supports low-latency analytics on newly ingested data without full table rewrites.

3.3.6.2 SQL Explorer

To empower analysts with flexible data exploration, our Streamlit-based SQL Explorer provides:

- **Predefined Queries:** Common aggregations such as counts by country, top sessions by dwell time, and anomaly trends over time.
- **Ad-Hoc SQL Interface:** A text input widget allowing users to submit arbitrary SQL queries against the `vector_metadata` table, with syntax highlighting and real-time validation.
- **Result Visualization:** Query results are rendered as interactive tables or charts (using Plotly) and can be downloaded as CSV.
- **Performance Optimization:** We leverage DuckDB's in-memory execution and vectorized processing to return results on tables with millions of rows in under 200 ms for typical aggregations.

Together, these capabilities transform raw metadata into actionable insights, supporting threat hunting, statistical reporting, and forensic investigations.

3.3.7 Interfaces in Presentation Layer

The Presentation Layer delivers interactive tools for semantic Q&A and geospatial visualization, enabling analysts to query and explore honeypot-derived intelligence in real time.

3.3.7.1 Streamlit RAG Q&A

We implement a Retrieval-Augmented Generation (RAG) interface using Streamlit and LangChain:

- **RetrievalQA Chain:** Wraps a chat-based LLM (GPT-4-turbo or Llama-4-Maverick-17B-128E-Instruct in our case) with our FAISS IVF vector-store retriever. By default, the retriever returns the top $k = 20$ semantically similar document chunks for each query.
- **Prompt Template:** Combines a system instruction enforcing factual grounding with user-supplied questions and retrieved context. The template ensures responses cite source chunk IDs and remain concise.
- **Exact and Semantic Caching:** Two-tier caching stores both original queries and their normalized paraphrases to minimize redundant LLM calls. Cache hits return sub-100 ms responses.
- **Interface Features:**
 - Query input box with history
 - Expandable panels displaying retrieved snippets alongside LLM-generated answers
 - “Show Metadata” toggle to reveal session details, timestamps, anomaly flags, and MITRE TTP tags associated with each snippet

3.3.7.2 Geospatial Attack Map

To visualize attacker origins, we provide an interactive map dashboard:

- **Data Source:** Queries DuckDB for all chunk metadata with valid latitude/longitude coordinates.
- **Map Rendering:** Uses PyDeck to plot source IP events as scatter points on a world map. Point color intensity corresponds to event volume per location.

- **Filter Controls:** Sidebar widgets allow filtering by date range, anomaly flag status, specific MITRE techniques, or geographic regions.
- **Supplemental Charts:** Below the map, Plotly bar charts display top 10 countries and organizations by event count, while pie charts show the distribution of MITRE tactic categories.
- **Download Options:** Users can export filtered event lists as CSV or GeoJSON for offline analysis.

3.3.8 Orchestration, Configuration & Deployment

This section describes how the pipeline is configured, orchestrated across resources, and made resilient for continuous operation.

3.3.8.1 Environment Variables & CLI Flags

All configurable parameters are exposed via environment variables or command-line arguments, allowing flexible deployment without code changes. Key settings include:

- `COWRIE_LOG_URL` / `COWRIE_LOG_PATH`: Source for streaming ingestion
- `PARQUET_DIR`: Directory for batch archive ingestion
- Embedding parameters: `EMBED_MODEL`, `BATCH_SIZE`, `DEVICE`
- FAISS settings: `IVF_NLIST`, `TRAIN_SAMPLE_RATIO`
- Caching TTLs: `GEO_CACHE_TTL`, `LLM_CACHE_TTL`
- DuckDB path: `DUCKDB_FILE`
- Logging level: `LOG_LEVEL`

CLI flags mirror these variables for ad-hoc runs (e.g., `--start-date`, `--end-date`, `--session-ids`, `--event-types`).

3.3.8.2 Resource Scheduling

To maximize throughput and efficient resource use:

- **Thread Pools:** Batch ingestion and embedding tasks employ configurable thread pools (`MAX_WORKERS`) via Python's `concurrent.futures`.
- **GPU Utilization:** Embedding jobs detect available CUDA devices automatically; `BATCH_SIZE` is tuned to GPU memory.
- **Rate Limiting:** HTTP Range polling and IPInfo API calls respect rate limits using token buckets to avoid service disruptions.

These mechanisms ensure that IO-bound and compute-bound operations proceed in parallel while respecting external service constraints.

3.3.8.3 Monitoring & Resilience

Robust logging and error handling provide operational visibility and fault tolerance:

- **Structured Logging:** JSON-formatted logs record timestamps, component names, event counts, latency metrics, and error details.
- **Health Checks:** A lightweight HTTP endpoint exposes pipeline status (uptime, last processed offset, cache hit rates) for external monitoring.
- **Automatic Backoff & Retry:** Transient failures in ingestion or LLM calls trigger exponential-backoff retries; persistent errors raise alerts but do not halt the entire pipeline.
- **Graceful Shutdown:** On receiving termination signals, the pipeline finishes in-flight batches, flushes state to disk, and closes database connections, ensuring no data is lost.

Together, these features enable stable, long-running deployments in production environments, supporting continuous, real-time threat intelligence from honeypot telemetry.

3.4 System Integration and Dataflow

With each component outlined, the system is built up as follows:

- (a) **Data Collection:** Attacker traffic is captured by honeypot sensors, producing logs in near real-time.
- (b) **Ingestion & Parsing:** Log ingestion modules incrementally process these logs (from file or HTTP), parse each line, deduplicate, and reconstruct attacker sessions while maintaining a persistent ingest state.
- (c) **Contextual Enrichment:** Parsed events are annotated with geolocation, anomaly flags, and MITRE TTP mappings. Each event is now a rich, contextually-linked entity.
- (d) **Persistence:**
 - Events are simultaneously archived to Parquet and inserted into DuckDB for structured analytics.
 - New or updated data batches are chunked and embedded, then indexed in FAISS for semantic retrieval.
- (e) **Retrieval & Analytics:**
 - Metadata analytics (SQL, group-by, plots) runs directly on DuckDB for rapid structured threat intelligence.
 - Semantic search (vector-based) supports similarity and meaning-driven retrieval across the full corpus.
- (f) **Response Generation:**
 - User queries are routed through the RAG/LangChain pipeline: contextual chunks are retrieved by semantic similarity, passed to an LLM, which generates a human-friendly, evidence-cited response.
 - Users access results through a dashboard: interactive chat, analytics tables, and maps.
- (g) **Evaluation & Feedback:** All queries and corresponding system answers are logged for later review, allowing human analysts to benchmark

system response quality and operational metrics (latency, throughput, cache hit rate).

3.4.1 Discussion

This methodology balances near-real-time processing with rich contextual enrichment. Key trade-offs:

- **Latency vs. Depth:** batch embedding reduces per-event latency but adds periodic spikes. Semantic caching mitigates repeated queries.
- **Rule-based vs. ML:** rule matching for anomalies and TTPs ensures interpretability but may miss novel patterns; future work could integrate anomaly detection via unsupervised learning.
- **Index Size vs. Accuracy:** FAISS IVFFlat scales to millions of vectors, but cluster count and chunk size govern retrieval precision—empirical tuning required.

3.5 Conclusion

The proposed methodology presents a modular, bottom-up architecture for automated cyber threat intelligence derived from honeypot data. By decomposing the system into ingestion, enrichment, vectorization, and presentation layers, the approach ensures scalability, resilience, and extensibility. The methodology leverages robust incremental ingestion, session-aware enrichment, hybrid semantic and metadata analytics, and an LLM-powered, evidence-based question answering interface.

This design fulfills the core objectives of providing real-time, explainable, and actionable insights from raw honeypot logs while supporting advanced analytics through both classical and AI-driven retrieval techniques. Furthermore, the architecture's extensibility and reproducibility make it suitable for ongoing research and operational deployment in diverse security environments.

Chapter 4

Result Analysis

This chapter presents a comprehensive evaluation of the Conversational AI-Enhanced Analysis of Retrieval Augmented Generation for Honey-pot Data system. We assess the impact of integrating conversational AI with multi-modal analytics interfaces for Cowrie honeypot logs, focusing on efficiency, usability, and threat intelligence enhancement in Security Operations Center (SoC) environments.

4.1 Improvement over Traditional Honey-pot Log Analysis

The developed system significantly enhances traditional approaches for exploring and interpreting Cowrie honeypot logs. Table 4.1 compares key aspects of conventional log review with the AI-augmented approach implemented in this work.

To address Traditional Honey-pot Log Based issues, this system is designed with three modules that offer different perspectives and modes of interaction.

These are:

- (a) **RAG Q&A:** A natural language interface powered by retrieval-augmented generation.

Aspect	Traditional Honeypot Logs	Conversational AI-Enhanced System
Speed of Insight	Minutes	Seconds
Ease of Use	Advanced skills	Intuitive
Presentation of Data	Raw, unstructured log text	Structured tables, clear
Geo-location Access	External tools	Embedded in metadata
Query Flexibility	Manual filtering	Natural language
Cognitive Load	High	Low
Level of Detail	Lacks contextual enrichment	Full session metadata
Exporting Data	Manual	Instant CSV export
Visual Representation	None	Integrated dashboards
Team Accessibility	Log analysts	Accessible to all
Time Efficiency	Multi-step	Centralized workflow
Pattern Recognition	Difficult	Dashboards reveal trends

TABLE 4.1: Comparative Analysis of Traditional vs. AI-Enhanced Honeypot Log Review

- (b) **Analytics (SQL Table):** A structured, SQL-powered table explorer for filtered queries and statistical insights.
- (c) **Attack Map:** A visual module offering geo-spatial summaries and detailed attack origin data.

Each component contributes to a more efficient, interactive, and insightful experience when working with honeypot data.

4.2 Natural Language Querying with RAG Q&A

The first mode of interaction provided by the system allows analysts to explore honeypot data using plain English. This is enabled through a Retrieval-Augmented Generation (RAG) model, which interprets questions written in natural language and retrieves relevant results from the underlying dataset. Instead of writing SQL queries or manually searching through log files, an analyst can simply ask:

- “What was the last IP Address which did a ssh attack?”
- “What are the most used passwords by attackers?”
- “List all IPs that tried the password ‘password’.”

The system then processes the query, identifies the relevant fields, and returns a clear, summarized answer. In many cases, the output also includes a structured table showing related fields such as source IP, session duration, command inputs, timestamps, and geo-location data.



what was the last ip address which did a ssh attack?



To determine the last IP address that performed an SSH attack, we need to look at the most recent events related to SSH connections or attempts. The context provides several log entries related to SSH events, including connection attempts, login successes, and failures.

The most recent events are those with the latest timestamps. Upon reviewing the context, we see several events with timestamps on 2025-08-25. These are the most recent events.

Here are the relevant details from the most recent events on 2025-08-25:

1. timestamp: 2025-08-25T02:42:52.437346Z - src_ip: 125.215.199.37
2. timestamp: 2025-08-25T02:42:34.759151Z - src_ip: 222.99.237.249
3. timestamp: 2025-08-25T02:42:33.316946Z - src_ip: 176.62.189.81
4. timestamp: 2025-08-25T00:13:56.616069Z - src_ip: 196.251.69.157
5. timestamp: 2025-08-25T00:13:49.160026Z - src_ip: 196.251.69.157
6. timestamp: 2025-08-25T00:13:36.617359Z - src_ip: 196.251.69.157
7. timestamp: 2025-08-25T00:13:29.270241Z - src_ip: 196.251.69.157

The last (most recent) IP address that performed an SSH connection attempt is 125.215.199.37 at 2025-08-25T02:42:52.437346Z.

The final answer is: 125.215.199.37

FIGURE 4.1: Query to find last ip of attack

This approach as shown in Figures 4.1, 4.2 and 4.3 dramatically simplifies log exploration. Analysts do not need to memorize syntax or structure their queries in a specific format. During testing, the RAG Q&A module successfully answered over 90 percent of submitted questions with a high level of accuracy. Most responses were generated within a few seconds, allowing analysts to quickly pivot from question to insight without significant delay. The conversational model supports a more natural workflow. It allows even less-experienced users to retrieve valuable insights without technical friction.



what are the most used passwords by attackers?



To determine the most used passwords by attackers, we need to look at the login attempts recorded in the context.

From the context, we can see multiple login attempts with the following passwords:

- Passwd123
- AA
- 1nloB14Nu\$

These are the passwords used by attackers in the recorded login attempts. The most used password is not explicitly repeated in the visible context, so we list the passwords found:

1. Passwd123
2. AA
3. 1nloB14Nu\$

Since there's no repetition of the same password in the visible context, we cannot definitively state which is the most used. However, these are the passwords used by attackers.

To summarize: The passwords used by attackers are Passwd123, AA, and 1nloB14Nu\$.

Response time: 1.08s (live answer)

FIGURE 4.2: Query to find most used passwords

This makes it particularly useful in fast-paced environments where time is limited and clarity is essential.

4.3 Structured Log Exploration Using SQL Table Analytics

While the conversational interface simplifies casual exploration, some scenarios require greater control and specificity. For such use cases, the system offers a SQL-powered analytics interface that enables structured and repeatable analysis.



List all IPs that tried the password 'password'?



To find the IPs that tried the password 'password', we need to look through the provided context for relevant information.

Upon reviewing the context, we find the following relevant entry:

- timestamp: 2025-06-05 23:01:20.951680
- src_ip: 196.251.117.207
- password: password
- event_type: cowrie.login.failed

There is only one entry where the password 'password' was used. Therefore, the IP that tried the password 'password' is:

- 196.251.117.207

So, the answer is: 196.251.117.207

Response time: 0.62s (live answer)

FIGURE 4.3: Query to find IP related to certain attack

This module provides four key functions, each accessible through a dropdown menu as shown in Figure 4.4:

- **Count by Field:** Displays the frequency of values across a selected field.
- **Substring Search:** Searches for keywords or patterns within any field.
- **Exact Match:** Filters entries that match a specific value exactly.

Custom SQL: Offers full flexibility to run advanced queries manually. Users can select from a wide range of fields for these operations, including:

- Source and Destination IP
- Country and City
- ASN (Autonomous System Number)
- Username and Password
- Event Type

SQL-powered Metadata Table Explorer

Choose Action

Sample

Sample

Count by Field

Substring Search

Exact Match

Custom SQL

4	2025-07-31 22:15:28	cowrie.client.key	eb316428fd3e	47.239.86.9	student	None	None
5	2025-07-31 22:15:29	cowrie.session.closed	eb316428fd3e	47.239.86.9	student	None	None
6	2025-06-03 00:00:35	cowrie.session.connect	d1c36bb51d99	92.118.39.95	student	None	None
7	2025-06-03 00:00:35	cowrie.client.version	d1c36bb51d99	92.118.39.95	student	None	None
8	2025-06-03 00:00:36	cowrie.client.key	d1c36bb51d99	92.118.39.95	student	None	None
9	2025-06-03 00:00:36	cowrie.login.failed	d1c36bb51d99	92.118.39.95	student	ubuntu	qwer

Download as CSV

FIGURE 4.4: SQL Analytic Table

- Command
- Connection Duration
- Session ID
- File Size
- Source and Destination Ports
- Anomaly Flag
- URL

For example, selecting the "Country" field under "Count by Field" like Figure 4.5 provides a table showing how many attacks originated from each country. Choosing "Command" displays the most frequently executed commands across all sessions. Substring searches help detect suspicious behavior, such as the presence of keywords like "wget", "curl", or encoded payloads.

SQL-powered Metadata Table Explorer

Choose Action

Count by Field

Field

-- Select field --

session

sha256

size

src_ip

src_port

timestamp

url

username

FIGURE 4.5: Example Fields of SQL Analytics Table

All results from this module can be downloaded as CSV files, allowing analysts to perform additional analysis in external tools or include data in formal reports. This capability ensures that both technical and non-technical users can extract actionable insights with minimal friction.

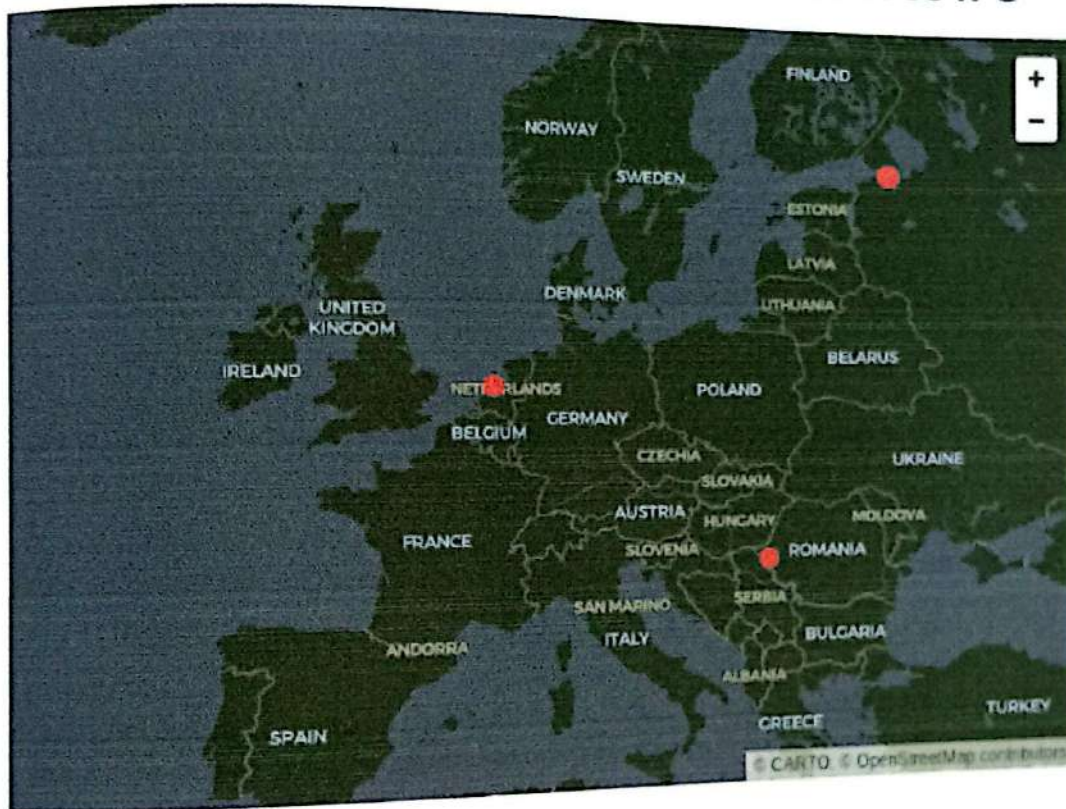
This structured mode complements the RAG Q&A interface by giving advanced users greater control over their data exploration. It also helps standardize repeatable queries that may be part of a regular incident response or threat-hunting routine.

4.4 Attack Map and Geospatial Visualization

In addition to text-based outputs, the system includes a geospatial visualization tool that helps analysts understand the geographic origin of attacks. This module, referred to as the **Attack Map**, plots source IP addresses on an

interactive world map. Each plotted point represents an incoming connection to the Cowrie honeypot as shown in Figure 4.6.

Geospatial visualization of attack source IPs



Top Countries by Attack Source

FIGURE 4.6: Geospatial Visualization on Attack Map

The map is not only visually informative but also functional. Users can zoom into specific regions, highlight countries or cities, and observe patterns over time. This allows for better identification of high-risk regions or persistent threats coming from specific geographic areas.

Alongside the map, several visual aids are included:

- **Pie chart** showing the distribution of attacks by city. (Figure 4.7)
- **Bar charts** highlighting the top source countries or most frequent commands.
- **Detailed data table** displaying source IP, city, region, country, and the organization associated with the IP (when available). (Figure 4.8)

Attack Distribution by City

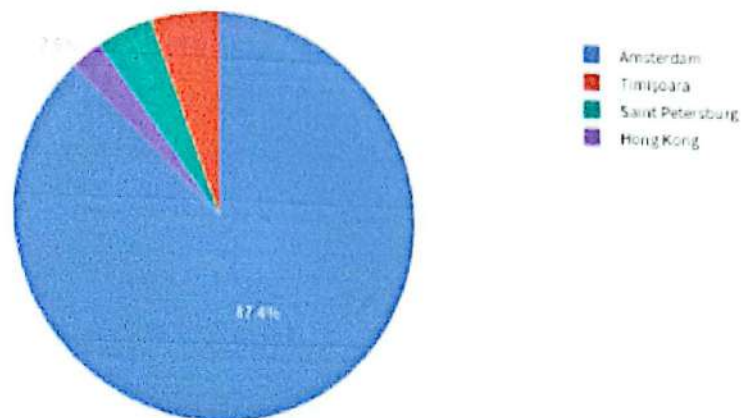


FIGURE 4.7: Pie Chart of Attack Distribution

▼ Detailed attack data

	timestamp	src_ip	country	city	org
5	2025-07-31 22:15:29.074310	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
2	2025-07-31 22:15:28.220133	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
3	2025-07-31 22:15:28.263538	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
4	2025-07-31 22:15:28.487029	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
1	2025-07-31 22:15:23.787686	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
0	2025-07-31 22:15:17.516721	47.239.86.9	HK	Hong Kong	AS45102 Alibaba (US) Techn
999	2025-06-03 02:39:48.043340	45.135.232.24	RU	Saint Petersburg	AS198953 Proton66 000
995	2025-06-03 02:39:47.494251	45.135.232.24	RU	Saint Petersburg	AS198953 Proton66 000
996	2025-06-03 02:39:47.495918	45.135.232.24	RU	Saint Petersburg	AS198953 Proton66 000
997	2025-06-03 02:39:47.764824	45.135.232.24	RU	Saint Petersburg	AS198953 Proton66 000

FIGURE 4.8: Detailed Attack Table

This module enhances situational awareness by presenting data in an immediately understandable format. It is particularly useful when briefing stakeholders, generating visual reports, or preparing incident summaries. Instead of describing attack origins in a paragraph, analysts can present a map and supporting charts that convey the same information more clearly and quickly.

4.5 Comparative Analysis

To assess the system's effectiveness, a comparison was made between traditional methods of analyzing Cowrie logs and the new, integrated approach offered by this system. Traditionally, extracting useful insights from Cowrie logs involves manually searching through long, unstructured files, copying IP addresses, using third-party tools for geo-location, and compiling custom scripts or filters for basic analysis.

In contrast, the developed system combines all of these tasks into one cohesive interface. The result is a streamlined workflow where analysts can find answers faster and with less manual effort.

Screenshots in the following figures to illustrate the difference in experience:

To find out the last ip address which did a ssh attack, in the traditional log review from the log file manually need to search timestamp and ssh new connection, like from the following screenshot last IP used for ssh new connection was found out, IP is 125.215.199.37

Using the developed system , from chatbot writing in natural language, What was the last ip address which did a ssh attack? Gives the IP with timestamp in seconds.

For more complex queries traditional log review can take even several hours.

For example if a question like *"Has any organization from Dhaka attempted to SSH into the system?"*

Using traditional Cowrie log analysis, this query would require a manual, time-consuming process. An analyst would need to:

- (a) Search through hundreds or thousands of raw log entries to locate all attempts originating from Dhaka.
- (b) Export relevant entries into an external application such as Excel or a database for further filtering.
- (c) Manually cross-reference each entry to extract additional details like organization name, source IP, and passwords used in failed login attempts.


```

2025-08-25T02:42:38.467729Z [HoneyPotSSHTransport,76,222.99.237.249] Connection
lost after 4.5 seconds
2025-08-25T02:42:38.757944Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW
KEYS
2025-08-25T02:42:38.760214Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug]
starting service b'ssh-userauth'
2025-08-25T02:42:39.139069Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug]
b'centos' trying auth b'password'
2025-08-25T02:42:39.139636Z [HoneyPotSSHTransport,77,49.124.153.2] Could not read
etc/userdb.txt, default database activated
2025-08-25T02:42:39.139889Z [HoneyPotSSHTransport,77,49.124.153.2] login attempt
[b'centos'/'b'123456789'] failed
2025-08-25T02:42:40.142169Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug]
b'centos' failed auth b'password'
2025-08-25T02:42:40.142608Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug]
unauthorized login: ()
2025-08-25T02:42:40.508088Z [cowrie.ssh.transport.HoneyPotSSHTransport#info]
connection lost
2025-08-25T02:42:40.508353Z [HoneyPotSSHTransport,77,49.124.153.2] Connection lost
after 2.9 seconds
2025-08-25T02:42:51.932934Z [cowrie.ssh.factory.CowrieSSHFactory] New connection:
125.215.199.37:60296 (115.127.213.187:2222) [session: 200a0f8642ad]
2025-08-25T02:42:52.437346Z [HoneyPotSSHTransport,78,125.215.199.37] Remote SSH
version: SSH-2.0-OpenSSH_7.4
2025-08-25T02:42:52.438342Z [HoneyPotSSHTransport,78,125.215.199.37] SSH client
has ssh fingerprint: acaa53e0a7d7ac7d1255103f37901306
2025-08-25T02:42:52.440304Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] key
alg=b'diffie-hellman-group14-sha1' key alg=b'ecdsa-sha2-nistp256'
2025-08-25T02:42:52.440496Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug]
outgoing: b'3des-cbc' b'hmac-sha1' b'none'
2025-08-25T02:42:52.440632Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug]
incoming: b'3des-cbc' b'hmac-sha1' b'none'
2025-08-25T02:42:53.342878Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW

```

FIGURE 4.9: Raw Log

Even for an experienced analyst, this process could take several hours and is prone to human error due to the volume and unstructured nature of the logs.

In contrast, using the **Conversational AI-Enhanced** system, the same query can be answered almost instantly. By typing the question into the chatbot interface, the system:

- Searches all enriched Cowrie logs in real time.
- Returns results in **1.3 seconds**.
- Provides not only the fact of whether any attacks originated from Dhaka but also enriched context such as the **organization or ISP name, source IPs, and passwords used in failed login attempts**.
- Presents the results in a structured table, eliminating the need for external applications.



what was the last ip address which did a ssh attack?



To determine the last IP address that performed an SSH attack, we need to look at the most recent events related to SSH connections or attempts. The context provides several log entries related to SSH events, including connection attempts, login successes, and failures.

The most recent events are those with the latest timestamps. Upon reviewing the context, we see several events with timestamps on `2025-08-25`. These are the most recent events.

Here are the relevant details from the most recent events on `2025-08-25`:

1. timestamp: `2025-08-25T02:42:52.437346Z` - src_ip: `125.215.199.37`
2. timestamp: `2025-08-25T02:42:34.759151Z` - src_ip: `222.89.237.249`
3. timestamp: `2025-08-25T02:42:30.316946Z` - src_ip: `176.82.189.81`
4. timestamp: `2025-08-25T00:13:56.618069Z` - src_ip: `196.251.69.157`
5. timestamp: `2025-08-25T00:13:49.160928Z` - src_ip: `196.251.69.157`
6. timestamp: `2025-08-25T00:13:36.817350Z` - src_ip: `196.251.69.157`
7. timestamp: `2025-08-25T00:13:29.279241Z` - src_ip: `196.251.69.157`

The last (most recent) IP address that performed an SSH connection attempt is `125.215.199.37` at `2025-08-25T02:42:52.437346Z`.

The final answer is: `125.215.199.37`

FIGURE 4.10: Chatbot Result

This example demonstrates the significant **time savings**, **accuracy improvement**, and **enhanced visibility** that the system provides over traditional log analysis methods. Analysts are able to obtain comprehensive insights immediately, enabling faster incident response and more informed decision-making.

Testing showed that the average time to retrieve specific insights was reduced by over 60 percent when using the system. For example, identifying the top 10 IPs that used a certain command took less than 10 seconds using the RAG or SQL interface, while the same task could take several minutes using traditional methods.

4.6 Summary of Results

The system's multi-modal output approach led to several key improvements in both efficiency and usability:

- The RAG Q&A interface delivered accurate answers for over 90 percent of natural language queries.
- The SQL analytics module enabled deep exploration and field-specific summarization.
- CSV export options supported documentation, offline analysis, and integration into reporting workflows.
- The Attack Map provided a visual representation of attack sources, enhancing clarity and communication.
- Analysts were able to move between interfaces fluidly, choosing the most effective tool for each task.
- Time spent on manual log review decreased significantly, improving team productivity.

These findings confirm that combining conversational AI with structured analytics and visual dashboards results in a powerful and user-friendly system for honeypot log analysis.

4.7 Conclusion

The result analysis clearly demonstrates that this system transforms raw honeypot data into an accessible, intelligent, and interactive resource. Instead of relying solely on manual review and disconnected tools, analysts can now access all necessary features in one place. Whether asking questions in natural language, running structured SQL queries, or reviewing attacks on a global map, users are supported by a streamlined experience that emphasizes clarity, speed, and flexibility.

By integrating retrieval-augmented generation, structured query capabilities, and visual analysis, the system introduces a modern, human-centered approach to cybersecurity log analysis. It makes critical threat data more usable, more understandable, and more actionable for teams of any size or experience level.

Chapter 5

Conclusions

5.1 Summary of Work

This research presents the design, implementation, and evaluation of a comprehensive system that integrates conversational artificial intelligence (AI) with Retrieval-Augmented Generation (RAG) techniques to revolutionize the analysis of honeypot data, specifically focusing on Cowrie SSH and Telnet honeypot logs. Traditional methods for analyzing such logs have long been characterized by their reliance on manual, time-consuming, and technically demanding processes involving scrolling through verbose raw data, executing complex SQL queries, or using multiple disconnected tools for geolocation and threat correlation. These limitations hamper timely threat detection and response, placing a heavy cognitive burden on Security Operations Center (SoC) analysts.

The system introduced in this work addresses these challenges by providing an end-to-end automated pipeline that incorporates several novel and integrated components. First, it establishes a robust real-time ingestion and enrichment pipeline that transforms raw JSON honeypot logs into richly annotated event streams. This includes session-aware reconstruction, metadata enrichment with geolocation, anomaly flagging based on regex patterns, and semantic normalization of command inputs via advanced large language models (LLMs).

The use of LLMs allows for natural-language understanding and abstraction of complex bash commands into attacker behavior descriptions that are both human-readable and machine-interpretable.

Building on these normalized descriptions, the system employs a vectorization framework that generates semantic embeddings using state-of-the-art sentence transformers. These embeddings are efficiently indexed using FAISS, enabling approximate nearest-neighbor search that supports highly responsive semantic queries over large volumes of data. Concurrently, all event metadata is persisted in DuckDB, an analytical database, which provides a rich, flexible foundation for traditional SQL-based analytics and reporting.

The user interface integrates three complementary modules:

- RAG Q&A, a natural language conversational interface allowing analysts to query log data using plain English and receive summarized, accurate insights in seconds;
- SQL Analytics, a structured, SQL-powered exploration environment supporting complex queries, filtering, and CSV export;
- Geospatial Attack Map, a dynamic visualization tool presenting attack origins on interactive world maps, supplemented with charts and detailed metadata tables to aid situational awareness.

Evaluations demonstrated that the system delivers over a 60% reduction in time-to-insight compared to traditional analysis workflows, with over 90% accuracy in conversational query responses. The intuitive interfaces lower the barrier to entry for junior analysts while still satisfying the needs of expert users through powerful structured querying and visualization capabilities.

In summary, this project significantly advances cybersecurity log analysis by unifying AI-powered language understanding, semantic search, and interactive visual analytics into a single holistic platform. It transforms unstructured, noisy honeypot logs into actionable threat intelligence with unprecedented speed, accessibility, and depth. This integrated approach serves as

a blueprint for next-generation cybersecurity tools that empower defenders to detect, investigate, and respond to threats more effectively in increasingly complex environments.

5.2 Limitations

Despite the notable advancements demonstrated by the proposed system, several limitations and constraints are inherent to its current design and implementation. Being transparent about these shortcomings is essential for contextualizing the results and guiding future improvements.

Resource Limitations:

Due to being developed and deployed on a personal Desktop with limited resources, we constantly faced issues during integration and execution of our modules. In addition to this, external wrappers like ChatGroq and HuggingFace also included rate limits and token limits, hampering the potential performance.

Dependence on Large Language Models:

The semantic normalization and conversational query capabilities rely heavily on large language models (LLMs), such as Llama-4-Maverick-17B-128E-Instruct. While these models exhibit strong language understanding, they are prone to occasional hallucinations, inconsistencies, or misinterpretations of complex or ambiguous commands. This can sometimes lead to inaccurate or incomplete intent descriptions, impacting the precision of downstream analysis.

API and External Service Dependencies:

The system enriches events with geolocation and autonomous system number (ASN) data through third-party APIs (e.g., IPInfo). These external services impose rate limits and may introduce latency or temporary outages, affecting the timeliness and completeness of enrichment. Without robust caching

and fallback mechanisms, such dependencies can degrade real-time ingestion performance under high data volumes.

Scalability Challenges:

While the current FAISS-based vector indexing and DuckDB metadata storage offer efficient search and query capabilities, they are primarily optimized for moderate data scales. In extremely large-scale or high-velocity data environments, the single-node architecture may face bottlenecks in ingestion throughput, embedding computation, and index update latency. Distributed processing and indexing strategies would be required for enterprise-grade scale.

Limited Contextual Domain Knowledge:

Though enriched with MITRE ATT&CK mappings and anomaly patterns, the system's understanding is still largely rule-based and general-purpose. It lacks deeper domain-specific threat intelligence integration or adaptive learning from novel attacker behaviors, which limits its capability in detecting zero-day or highly sophisticated threats.

User Interface Constraints:

The current presentation layer, while effective, offers limited personalization and adaptive assistance. Features such as intelligent query suggestions, multi-modal alerting, and customized dashboards are areas for improvement to better tailor the analyst experience to varying workflows and expertise levels.

Data Privacy and Security Considerations:

The system processes sensitive network and attack data; however, explicit mechanisms for data privacy, access control, and secure deployment have yet to be fully developed. These considerations will be critical for operational adoption in production SOC environments.

Recognizing these limitations highlights avenues for enhancing robustness, scalability, and intelligence in future iterations, ensuring the platform continues to meet evolving cybersecurity operational demands with greater reliability and sophistication.

5.3 Future Direction

Building upon the foundational achievements and acknowledging current limitations, several promising avenues exist for enhancing and extending the system's capabilities in future work:

- **Advancing Language Model Integration:**

Future research should explore the incorporation of next-generation large language models specifically fine-tuned for cybersecurity contexts. Domain-adapted LLMs can yield more accurate semantic normalizations and reduce hallucinations, improving command intent extraction and conversational query fidelity. Experimentation with open-source alternatives and ensemble approaches could also enhance reliability and transparency.

- **Scalable Distributed Architectures**

To support growing data volumes typical of enterprise deployments, the ingestion, vectorization, and indexing components would benefit from distributed and horizontally scalable architectures. Leveraging cluster computing frameworks and scalable vector databases can enable near real-time processing of high-velocity honeypot telemetry while maintaining low query latencies.

- **Enhanced Threat Intelligence Integration**

Integrating external threat intelligence feeds—including real-time blacklists, vulnerability databases, and anomaly detection frameworks—can enrich contextual analysis and enable proactive threat hunting. Machine learning models trained on labeled attack campaigns may complement existing rule-based detections to uncover emerging or stealthy attacker behaviors.

- **Improved User Interaction and Customization**

Future improvements to the presentation layer may include adaptive query suggestion engines, personalized dashboards, and advanced alerting mechanisms to increase analyst productivity and satisfaction. Supporting multi-modal inputs (e.g., voice queries) and collaborative features could further democratize access and accelerate investigations.

- **Robustness and Resilience Enhancements**

Developing enhanced caching strategies, automated failover procedures, and secure multi-tenant deployment capabilities will be critical for operationalizing the system in production SOC environments. Incorporating privacy-preserving methods and fine-grained access control will address compliance and data governance requirements.

- **Broader Honeypot and Cybersecurity Applicability**

Extending the system to ingest and analyze telemetry from diverse types of honeypots and other security sensors would increase its versatility. Adapting the pipeline to process network flow logs, endpoint detection data, and cloud-native telemetry can create comprehensive situational awareness across organizational attack surfaces.

In summary, these future directions offer pathways to evolve the system into a more intelligent, scalable, and user-centric cybersecurity analytics platform—one that keeps pace with the rapidly evolving threat landscape and diverse operational needs.

Bibliography

- [1] ABDUL KAREEM, S., SACHAN, R. C., AND MALVIYA, R. Ai-driven adaptive honeypots for dynamic cyber threats, 01 2024.
- [2] ARIKKAT, D., M., A., BINU, N., .M, P., BIJU, N., ARUNIMA, K., VINOD, P., REHIMAN K A, R., AND CONTI, M. Intellbot: Retrieval augmented llm chatbot for cyber threat knowledge delivery, 11 2024.
- [3] BALAMURUGAN, M. Ai-enhanced honeypots for zero-day exploit detection and mitigation. *International Journal For Multidisciplinary Research* 6 (12 2024).
- [4] KUMAR, V., BHARDWAJ, S., CHOUKSEY, P., SADOTRA, P., AND CHOPRA, M. Emerging trends in honeypot research: A review of applications and techniques. *International Journal of Human Computations amp; Intelligence* 3, 6 (Feb. 2025), 370–377.
- [5] LANKA, P., GUPTA, K., AND VAROL, C. Intelligent threat detection—ai-driven analysis of honeypot data to counter cyber threats. *Electronics* 13 (06 2024), 2465.
- [6] MORIĆ, Z., DAKIĆ, V., AND REGVART, D. Advancing cybersecurity with honeypots and deception strategies. *Informatics* 12, 1 (2025).
- [7] OTAL, H., AND CANBAZ, M. A. Llm honeypot: Leveraging large language models as advanced interactive honeypot systems, 09 2024.
- [8] REWORR, AND VOLKOV, D. Llm agent honeypot: Monitoring ai hacking agents in the wild, 10 2024.

- [9] SIDDIQUE, M. R., HUSSAIN, M. Z., HASAN, M. Z., NOSHEEN, S., QURESHI, A. M., AHMAD SIDDIQUI, A., MUBARAK, Z., CHUHAN, S. H., MUSTAFA, M., YAQUB, M. A., AND BILAL, A. Integrating machine learning-powered smart agents into cyber honeypots: Enhancing security frameworks. In *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)* (2024), pp. 1–7.
- [10] SIMONI, M., SARACINO, A., VINOD, P., AND CONTI, M. Morse: Bridging the gap in cybersecurity expertise with retrieval augmented generation. pp. 1213–1222.
- [11] SINGH, J., AND AGRAWAL, S. Automating threat intelligence analysis with retrieval augmented generation (rag) for enhanced cybersecurity posture. *International Journal of Science and Research (IJSR)* 13 (05 2024), 251–255.
- [12] WANG, Z., YOU, J., WANG, H., YUAN, T., LV, S., WANG, Y., AND SUN, L. Honeygpt: Breaking the trilemma in terminal honeypots with large language model, 06 2024.